

Football Tournament Management System

Developer Manual

Technical Documentation for Developers

Author: Rostand Cedrix Assonfo Demaboi
Institution: Technische Hochschule Georg Agricola, Bochum
Module: Introduction to Database Management Systems
Semester: Summer Term 2026

Version 1.0

September 10, 2026

Contents

1	Introduction	2
1.1	Purpose of this Document	2
1.2	System Overview	2
1.3	Main Functional Areas	2
1.4	Important Implementation Note	2
2	Technology Stack	3
2.1	Backend	3
2.2	Frontend	3
2.3	Database	3
2.4	Docker	3
3	System Architecture	4
3.1	Architecture Overview	4
3.2	Frontend Layer	4
3.3	Backend Layer	4
3.4	Database Layer	5
4	Project Structure	5
4.1	Backend Modules	6
4.1.1	main.py	6
4.1.2	database.py	6
4.1.3	auth.py	6
4.1.4	models/	6
4.1.5	schemas/	6
4.1.6	routes/	6
5	Installation and Configuration	6
5.1	Prerequisites	6
5.2	Backend Virtual Environment	7
5.3	Database Configuration	7
5.4	Database Initialization	7
6	Docker and Deployment	8
6.1	Backend Image	8
6.2	Starting Docker Compose	8
6.3	Current Compose Limitation	8
7	Database Design	8
7.1	Database Overview	8
7.2	Entity Relationship Diagram	9
7.3	Users Table	9
7.4	Tournaments Table	9
7.5	Teams Table	10
7.6	Players Table	10

7.7	Matches Table	10
7.8	Team/Tournament Association	11
8	REST API	11
8.1	Base URL	11
8.2	Health Endpoint	11
8.3	Tournament Endpoints	11
8.4	Team Endpoints	12
8.5	Match Endpoints	12
8.6	Ranking Endpoint	12
9	Frontend API Client	12
9.1	api.py	12
10	Backend Implementation	13
10.1	Application Entry Point	13
10.2	Database Session	13
10.3	Pydantic Schemas	14
11	Frontend Architecture	14
11.1	Connection Dialog	14
11.2	Dashboard	14
11.3	Main Application	15
12	Business Logic	15
12.1	Tournament	15
12.2	Match	15
12.3	Score	15
12.4	Standings	16
13	Authentication and Authorization	16
13.1	Token Verification	16
13.2	Current Integration Status	16
13.3	Recommended Authorization Model	16
14	Error Handling	17
15	Database Integrity	17
15.1	Primary Keys	17
15.2	Unique Constraints	17
15.3	Foreign Keys	18
15.4	Match Integrity	18
16	Testing	18
16.1	Current Testing Status	18
16.2	Recommended Unit Tests	18
16.3	API Tests	19
16.4	Database Tests	19

17 Development Workflow	19
18 Troubleshooting	20
18.1 Backend Does Not Start	20
18.2 Database Connection Fails	20
18.3 Frontend Cannot Connect	20
18.4 Authentication Fails	21
19 Known Issues and Technical Debt	21
19.1 Authentication Header Inconsistency	21
19.2 Authentication Dependency Not Attached	21
19.3 Ranking Endpoint Missing	21
19.4 Frontend Ranking Function Name	21
19.5 Duplicate API Function	21
19.6 Seed Role Inconsistency	21
19.7 Docker Compose Database	22
19.8 Secrets	22
20 Extending the Application	22
20.1 Adding a New Feature	22
21 Deployment	23
21.1 Development Environment	23
21.2 Production Considerations	23
22 Appendix	23
22.1 Useful Commands	23
22.2 Default Development Configuration	24
22.3 Compiling this Manual	24

1 Introduction

1.1 Purpose of this Document

This document is the technical developer manual for the Football Tournament Management System.

Its purpose is to provide future developers with the information required to understand, install, maintain, test, and extend the application.

Unlike a user manual, which focuses on how to operate the application, this document focuses on the internal architecture, source code, database structure, REST API, business logic, configuration, and development workflow.

1.2 System Overview

The Football Tournament Management System is a small client-server application designed to organize football competitions.

The application is divided into three principal layers:

- a Python/Tkinter desktop frontend;
- a Python/FastAPI REST backend;
- a PostgreSQL relational database.

The system manages tournaments, teams, players, scheduled matches, match scores, referees, and tournament participation.

1.3 Main Functional Areas

The application provides functionality for:

- tournament management;
- team management;
- player storage;
- match scheduling;
- match score updates;
- tournament/team associations;
- tournament standings in the frontend.

1.4 Important Implementation Note

This manual documents the implementation found in the project source. Some parts of the README and frontend refer to functionality that is not completely connected to the current backend implementation. These differences are documented in [Section 19](#) instead of being hidden.

2 Technology Stack

Component	Technology
Backend	Python 3.12 / FastAPI
ORM	SQLAlchemy
Validation	Pydantic
Frontend	Python / Tkinter
HTTP Client	Requests
Database	PostgreSQL
API Documentation	FastAPI / OpenAPI
Containerization	Docker / Docker Compose
Documentation	LaTeX

Table 1: Technology stack

2.1 Backend

The backend is implemented with FastAPI. It exposes REST endpoints for tournaments, teams, and matches.

SQLAlchemy is used for database access and ORM models, while Pydantic schemas are used for request and response validation.

2.2 Frontend

The frontend is a Python desktop application based on Tkinter. It communicates with the backend through HTTP requests.

The frontend contains a connection dialog, a dashboard, tournament, team, match, and standings views, as well as forms for administrative operations.

2.3 Database

The SQL database is PostgreSQL. The database initialization scripts are located in the `database/` directory.

2.4 Docker

The backend contains a Dockerfile based on Python 3.12 slim. The repository also contains a `docker-compose.yml` file.

The current Compose configuration should be checked before deployment, because the repository's Compose file does not define a PostgreSQL service even though PostgreSQL is required by the application.

3 System Architecture

3.1 Architecture Overview

The application follows a client-server architecture.

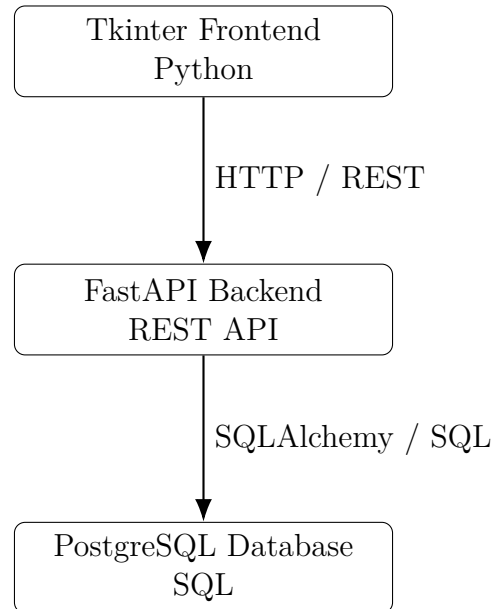


Figure 1: System architecture

3.2 Frontend Layer

The frontend is responsible for user interaction and presentation. It should communicate with the backend API rather than accessing the database directly.

The main frontend modules are located in:

```
1 frontend/src/football_tournament/
```

Important modules include:

- `api.py`: HTTP client and API helper functions;
- `connection_dialog.py`: API URL/token configuration;
- `ui/dashboard.py`: main dashboard and tabs;
- `__main__.py`: application entry point and forms.

3.3 Backend Layer

The backend is responsible for:

- receiving HTTP requests;
- validating request data;

- executing application logic;
- interacting with PostgreSQL;
- returning JSON responses.

The main backend package is:

```
1 backend/app/
```

3.4 Database Layer

The database stores persistent information and enforces structural integrity through primary keys, foreign keys, unique constraints, checks, and deletion rules.

4 Project Structure

The principal repository structure is:

```
1 football-tournament-manager/  
2 |  
3 +-- backend/  
4 |   +-- app/  
5 |     |   +-- __init__.py  
6 |     |   +-- auth.py  
7 |     |   +-- database.py  
8 |     |   +-- main.py  
9 |     |   +-- models/  
10 |    |   +-- routes/  
11 |    |   +-- schemas/  
12 |    +-- Dockerfile  
13 |    +-- requirements.txt  
14 |    +-- seed.py  
15 |    +-- .env  
16 |  
17 +-- frontend/  
18 |   +-- src/  
19 |       +-- football_tournament/  
20 |           +-- __main__.py  
21 |           +-- api.py  
22 |           +-- connection_dialog.py  
23 |           +-- ui/  
24 |  
25 +-- database/  
26 |   +-- init.sql  
27 |   +-- seed.sql  
28 |  
29 +-- docker-compose.yml  
30 +-- README.md  
31 +-- DEVELOPER_MANUAL.tex
```

4.1 Backend Modules

4.1.1 `main.py`

Creates the FastAPI application, configures CORS, and includes the tournament, team, and match routers.

4.1.2 `database.py`

Loads the database configuration, creates the SQLAlchemy engine and session factory, and provides database access.

The application expects the environment variable `DATABASE_URL`. A fallback PostgreSQL URL is also defined.

4.1.3 `auth.py`

Contains the API token verification dependency.

The current implementation expects the HTTP header `X-API-Token` and compares it with the `API_TOKEN` environment variable.

4.1.4 `models/`

Contains the SQLAlchemy ORM models for users, tournaments, teams, players, matches, and team/tournament associations.

4.1.5 `schemas/`

Contains the Pydantic schemas used by the API.

4.1.6 `routes/`

Contains the route modules:

- `tournaments.py`;
- `teams.py`;
- `matches.py`.

5 Installation and Configuration

5.1 Prerequisites

The development environment requires:

- Python 3.12 or compatible Python version;
- PostgreSQL;
- Docker and Docker Compose, if containerization is used;

- Git;
- Tkinter for the desktop frontend;
- LaTeX, if this developer manual is to be compiled.

5.2 Backend Virtual Environment

A virtual environment can be created with:

```
1 cd backend
2 python3 -m venv venv
3 source venv/bin/activate
4 pip install -r requirements.txt
```

5.3 Database Configuration

The backend uses environment variables for database configuration.

The repository currently contains a `backend/.env` file. For security, real database passwords must not be committed to source control.

The application code expects:

```
1 DATABASE_URL
```

The checked-in environment file also contains separate values such as database host, database name, user, password, and port.

A recommended development configuration is:

```
1 DATABASE_URL=postgresql://USER:PASSWORD@localhost:5432/
  football_db
2 API_TOKEN=change-this-development-token
```

5.4 Database Initialization

The SQL initialization script is:

```
1 database/init.sql
```

The test data script is:

```
1 database/seed.sql
```

The backend also contains a Python seed script:

```
1 backend/seed.py
```

The SQL seed and Python seed should be kept consistent with the database constraints.

6 Docker and Deployment

6.1 Backend Image

The backend Dockerfile uses Python 3.12 slim and starts Uvicorn.

The container exposes port 8000.

6.2 Starting Docker Compose

Use:

```
1 docker compose up --build -d
```

Check running services:

```
1 docker compose ps
```

View logs:

```
1 docker compose logs
```

Stop services:

```
1 docker compose down
```

6.3 Current Compose Limitation

The current `docker-compose.yml` contains the backend service, but does not define a PostgreSQL container.

Therefore PostgreSQL must currently be provided separately unless the Compose configuration is extended.

7 Database Design

7.1 Database Overview

The database contains six principal tables:

- users;
- tournaments;
- teams;
- players;
- matches;
- team_tournaments.

7.2 Entity Relationship Diagram

The following diagram represents the important relationships between the entities. In particular, a match references a team twice: once through `home_team_id` and once through `away_team_id`. These are deliberately shown as two separate relationships.

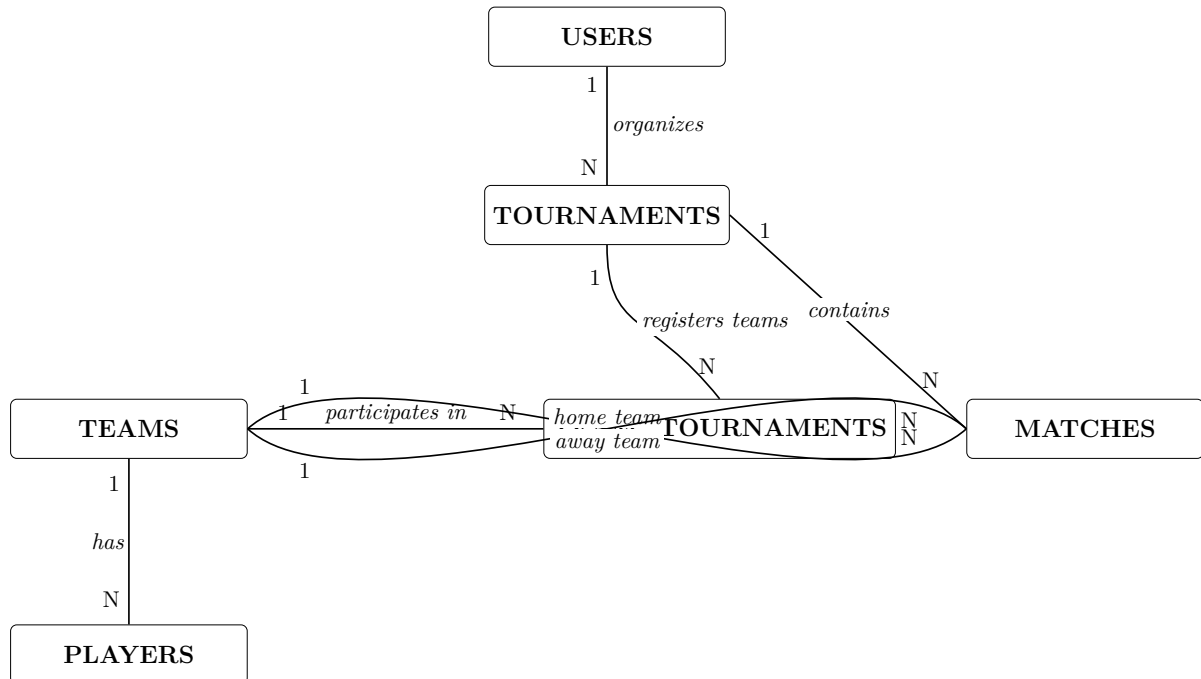


Figure 2: Entity relationship diagram with home-team and away-team relationships

7.3 Users Table

Column	Type	Constraint	Description
user_id	INTEGER	Primary key, identity	User identifier
username	VARCHAR	UNIQUE, NOT NULL	Login/user name
email	VARCHAR	UNIQUE, NOT NULL	Email address
password	VARCHAR	NOT NULL	Stored password value
role	VARCHAR	CHECK, NOT NULL	organizer, player, or viewer
created_at	TIMESTAMPTZ	NOT NULL	Account creation timestamp

7.4 Tournaments Table

Column	Type	Constraint	Description
tournament_id	INTEGER	Primary key, identity	Tournament identifier
name	VARCHAR	NOT NULL	Tournament name
start_date	DATE	NOT NULL	Start date
end_date	DATE	NOT NULL	End date
organizer_id	INTEGER	Foreign key	Responsible user

The database checks that the start date is not later than the end date. The organizer foreign key uses restrictive deletion behavior.

7.5 Teams Table

Column	Type	Constraint	Description
team_id	INTEGER	Primary key, identity	Team identifier
name	VARCHAR	UNIQUE, NOT NULL	Team name
coach	VARCHAR	Optional	Coach name

7.6 Players Table

Column	Type	Constraint	Description
player_id	INTEGER	Primary key, identity	Player identifier
first_name	VARCHAR	NOT NULL	First name
last_name	VARCHAR	NOT NULL	Last name
position	VARCHAR	Optional	Playing position
team_id	INTEGER	Foreign key	Player's team

The player/team relationship is one-to-many: one team can have many players. The player is deleted when its team is deleted.

7.7 Matches Table

Column	Type	Constraint	Description
match_id	INTEGER	Primary key, identity	Match identifier
match_date	TIMESTAMPZ	NOT NULL	Scheduled match date/time
home_score	INTEGER	NOT NULL, ≥ 0	Home score
away_score	INTEGER	NOT NULL, ≥ 0	Away score
tournament_id	INTEGER	Foreign key	Tournament containing match
home_team_id	INTEGER	Foreign key	Home team

Column	Type	Constraint	Description
away_team_id	INTEGER	Foreign key	Away team
referee	VARCHAR	Optional	Referee name

The database explicitly prevents a team from playing against itself by checking:

```
1 CHECK (home_team_id != away_team_id)
```

The two foreign keys from `matches` to `teams` are semantically different:

- `home_team_id`: team playing at home;
- `away_team_id`: team playing away.

7.8 Team/Tournament Association

The `team_tournaments` table represents the many-to-many relationship between teams and tournaments.

Its primary key is composed of:

```
1 (team_id, tournament_id)
```

This prevents the same team from being registered twice in the same tournament.

8 REST API

8.1 Base URL

The default development API URL used by the frontend is:

```
1 http://localhost:8000
```

FastAPI also provides interactive documentation at:

```
1 http://localhost:8000/docs
```

8.2 Health Endpoint

The backend exposes a health endpoint:

```
1 GET /health
```

A root endpoint is also available:

```
1 GET /
```

8.3 Tournament Endpoints

Method	Endpoint	Description
GET	/tournaments	List tournaments
GET	/tournaments/{id}	Retrieve one tournament
POST	/tournaments	Create tournament
DELETE	/tournaments/{id}	Delete tournament

The current backend does not expose the PUT tournament endpoint described in some project documentation.

8.4 Team Endpoints

Method	Endpoint	Description
GET	/teams	List teams
POST	/teams	Create team
DELETE	/teams/{id}	Delete team

8.5 Match Endpoints

Method	Endpoint	Description
GET	/matches	List matches
POST	/matches	Create match
PUT	/matches/{id}/score	Update match score
DELETE	/matches/{id}	Delete match

8.6 Ranking Endpoint

The frontend currently contains a function intended to request:

```
1 GET /tournaments/{tournament_id}/ranking
```

However, no corresponding ranking route is currently implemented in `backend/app/routes/tournament`. This is therefore a known integration gap.

9 Frontend API Client

9.1 api.py

The file:

```
1 frontend/src/football_tournament/api.py
```

centralizes HTTP communication with the backend.

The default configuration contains:

```
1 API_URL = "http://localhost:8000"
2 API_KEY = "your-secret-api-token-here"
```

The client sends the token through the header:

```
1 x-api-token
```

The client provides helper functions for:

- retrieving tournaments;
- creating tournaments;
- retrieving teams;
- creating teams;
- retrieving matches;
- creating matches;
- updating scores;
- deleting resources.

The module currently contains duplicate `create_match_scheduled` definitions and a compatibility-style `update_match_result` function with several fallback requests. These should be simplified during maintenance.

10 Backend Implementation

10.1 Application Entry Point

The FastAPI application is created in:

```
1 backend/app/main.py
```

The application configures CORS with all origins, methods, and headers in the current development implementation.

10.2 Database Session

Database access is centralized through SQLAlchemy.

The application creates a database engine and session factory in `database.py`.

Future changes should preserve the separation between:

1. HTTP request handling;
2. application logic;
3. database operations.

10.3 Pydantic Schemas

The schemas module contains definitions for:

- users;
- tournaments;
- teams;
- players;
- matches;
- team/tournament associations;
- rankings.

The ranking schema contains fields including position, team name, matches played, goals scored, goals conceded, and points.

11 Frontend Architecture

11.1 Connection Dialog

The connection dialog allows the user to enter:

- the API URL;
- the API token.

The token field is labelled as an X-API-Token value.

11.2 Dashboard

The dashboard is implemented in:

```
1 frontend/src/football_tournament/ui/dashboard.py
```

It contains tabs for:

- tournaments;
- standings;
- teams;
- matches.

11.3 Main Application

The main application and forms are defined in:

```
1 frontend/src/football_tournament/__main__.py
```

The application contains forms for scheduling matches and entering match results.

12 Business Logic

12.1 Tournament

A tournament has:

- a name;
- a start date;
- an end date;
- an organizer;
- participating teams through `team_tournaments`.

12.2 Match

A match belongs to one tournament and references two different teams.

The two team references are:

```
1 home_team_id  
2 away_team_id
```

This distinction is important when implementing score updates and ranking calculations.

12.3 Score

A match score contains:

- home-team goals;
- away-team goals.

The SQL schema requires non-negative scores.

12.4 Standings

The frontend and schema contain support for tournament standings. The ranking data structure contains:

- position;
- team name;
- matches played;
- goals scored;
- goals conceded;
- points.

A complete backend ranking endpoint is not currently present, so the standings integration should be completed before relying on it as a fully supported API feature.

13 Authentication and Authorization

13.1 Token Verification

The authentication helper is located in:

```
1 backend/app/auth.py
```

It expects:

```
1 X-API-Token: <API-TOKEN>
```

The token is compared with the value of the `API_TOKEN` environment variable.

13.2 Current Integration Status

Although the authentication dependency exists, the current `main.py` router configuration does not attach `verify_token` to the tournament, team, and match routes.

Therefore the code contains the authentication mechanism, but the current routes are not consistently protected by it.

This should be corrected before production deployment.

13.3 Recommended Authorization Model

A future implementation should explicitly define:

Operation	Viewer	Organizer
Read tournaments	Yes	Yes
Read teams	Yes	Yes
Read matches	Yes	Yes
Read standings	Yes	Yes
Create data	No	Yes
Update data	No	Yes
Delete data	No	Yes

Table 10: Recommended authorization matrix

14 Error Handling

The backend should use standard HTTP status codes.

Status	Meaning
200	Successful request
201	Resource successfully created
400	Invalid request
401	Unauthorized
404	Resource not found
409	Resource or database conflict
500	Internal server error

Table 11: Recommended HTTP status codes

The exact response behavior should be verified when new endpoints are implemented.

15 Database Integrity

The database schema contains several important integrity rules.

15.1 Primary Keys

All principal tables use identity-based integer primary keys, while `team_tournaments` uses a composite primary key.

15.2 Unique Constraints

The schema enforces uniqueness for:

- usernames;
- email addresses;
- team names.

15.3 Foreign Keys

Important foreign key relationships include:

- tournaments → users;
- players → teams;
- matches → tournaments;
- matches → home teams;
- matches → away teams;
- team/tournament associations → teams;
- team/tournament associations → tournaments.

15.4 Match Integrity

The database prevents a match from using the same team for both home and away positions.

```
1 CHECK (home_team_id != away_team_id)
```

This is preferable to relying only on frontend validation.

16 Testing

16.1 Current Testing Status

No dedicated automated test suite was found in the inspected project source. Consequently, tests should be added before major extensions or production deployment.

16.2 Recommended Unit Tests

Unit tests should cover:

- tournament validation;
- team validation;
- match validation;
- score validation;
- ranking calculations.

Example:

```
1 def test_goal_difference():
2     assert calculate_goal_difference(3, 1) == 2
```

16.3 API Tests

API tests should verify:

- successful requests;
- invalid requests;
- missing resources;
- score updates;
- database conflicts;
- authentication once it is attached to the routes.

16.4 Database Tests

Database tests should verify:

- primary keys;
- foreign keys;
- uniqueness constraints;
- score checks;
- home/away team checks;
- deletion behavior.

17 Development Workflow

A recommended development workflow is:

1. Pull the latest repository version.
2. Create a dedicated feature branch.
3. Modify the database schema if necessary.
4. Update SQLAlchemy models.
5. Update Pydantic schemas.
6. Implement or modify API routes.
7. Update the frontend API client.
8. Update the Tkinter interface.
9. Run tests.

10. Verify database integrity.
11. Update this manual.
12. Commit the changes.

Example:

```
1 git checkout -b feature/new-functionality
2
3 # Implement changes
4
5 git add .
6 git commit -m "Add new functionality"
7 git push origin feature/new-functionality
```

18 Troubleshooting

18.1 Backend Does Not Start

First verify the environment variables and database connection.

Then inspect Docker logs if Docker is being used:

```
1 docker compose ps
2 docker compose logs
```

18.2 Database Connection Fails

Check:

- PostgreSQL is running;
- the database exists;
- the username is correct;
- the password is correct;
- port 5432 is reachable;
- DATABASE_URL is correctly configured.

18.3 Frontend Cannot Connect

Verify that the backend is reachable at:

```
1 http://localhost:8000
```

Then check the API URL configured in the connection dialog.

18.4 Authentication Fails

The current authentication implementation expects:

```
1 X-API-Token
```

Make sure the frontend and backend use the same token and header name.

19 Known Issues and Technical Debt

The following items should be considered when maintaining the project.

19.1 Authentication Header Inconsistency

The source code uses `X-API-Token`, while some older documentation refers to `X-API-Key`. The implementation and documentation should use one name consistently.

19.2 Authentication Dependency Not Attached

The authentication helper exists but is not currently applied to the main CRUD routers. This should be fixed before deployment.

19.3 Ranking Endpoint Missing

The frontend expects a ranking endpoint, but the backend does not currently expose the corresponding route.

19.4 Frontend Ranking Function Name

The dashboard checks for a function named `get_tournament_ranking`, while the API client contains `get_standings`. These names should be unified.

19.5 Duplicate API Function

`create_match_scheduled` is defined more than once in the frontend API client. Duplicate definitions should be removed.

19.6 Seed Role Inconsistency

The SQL schema restricts user roles to:

```
1 organizer
2 player
3 viewer
```

The Python seed script currently uses `admin` for a test user. This conflicts with the SQL role constraint and should be corrected.

19.7 Docker Compose Database

The current Compose configuration does not provide a PostgreSQL service, so the database setup is not fully self-contained.

19.8 Secrets

A database password and development credentials are present in the repository archive. Real credentials should be rotated and moved to environment-specific secret storage.

20 Extending the Application

Potential future extensions include:

- complete player management;
- cards and disciplinary events;
- stadium management;
- player statistics;
- tournament groups;
- notifications;
- multiple competition formats;
- a complete ranking API.

20.1 Adding a New Feature

A database-backed feature should normally follow this sequence:

1. Modify the SQL schema if necessary.
2. Update the SQLAlchemy model.
3. Update Pydantic schemas.
4. Implement business logic.
5. Add or modify REST endpoints.
6. Update the frontend API client.
7. Update the Tkinter interface.
8. Add tests.
9. Update the developer manual.

21 Deployment

21.1 Development Environment

The intended development environment consists of:

- PostgreSQL;
- the FastAPI backend;
- the Tkinter frontend;
- optional Docker containers.

21.2 Production Considerations

Before production deployment, the following items should be addressed:

- protect all write endpoints;
- replace development API tokens;
- remove real secrets from source control;
- restrict CORS origins;
- provide a managed PostgreSQL instance;
- implement automated tests;
- implement database migrations;
- define backups;
- define monitoring and logging.

22 Appendix

22.1 Useful Commands

Command	Purpose
<code>docker compose up -build -d</code>	Start containers
<code>docker compose ps</code>	Show running containers
<code>docker compose logs</code>	Show service logs
<code>docker compose down</code>	Stop containers
<code>python3 -m venv venv</code>	Create Python virtual environment
<code>pip install -r requirements.txt</code>	Install Python dependencies

22.2 Default Development Configuration

Parameter	Value
API URL	<code>http://localhost:8000</code>
API documentation	<code>http://localhost:8000/docs</code>
Authentication header	<code>X-API-Token</code>
Database	PostgreSQL
Backend port	8000

22.3 Compiling this Manual

With a LaTeX installation, run twice:

```
1 pdflatex DEVELOPER_MANUAL.tex
2 pdflatex DEVELOPER_MANUAL.tex
```

The second compilation updates the table of contents and references.

With VS Code, the LaTeX Workshop extension can be used to build the document automatically.